LA RAPPRESENTAZIONE DELL'INFORMAZIONE

LUCA DIDACI
didaci@unica.it

La rappresentazione dell'informazione

- Rappresentazione dei numeri interi
- Il bit, il byte e i suoi multipli
- Rappresentazione dei caratteri

L’informazione

Gli odierni sistemi per l’elaborazione dell’informazione sono formati da elementi che possono assumere due soli stati:

0 o **1**.

È possibile rappresentare l’informazione utilizzando tali elementi?

…	1	0	1	1	0	1	1	0	1	0	1

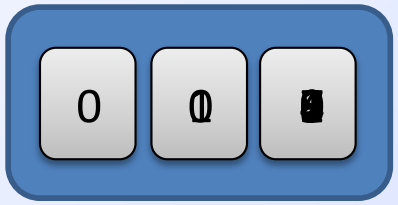
Esaminiamo la possibilità di ***rappresentare numeri interi***.

In seguito vedremo come è possibile rappresentare informazioni più complesse: testi, immagini, musica, ecc.

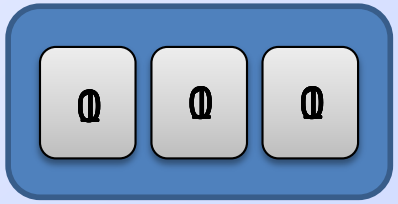
Vedremo anche come è possibile *elaborare*, e non solo *rappresentare*, tali informazioni.

Rappresentazione dei numeri interi

- Esempio: contatore con dieci simboli …



- … e contatore con due simboli



Rappresentazione dei numeri interi

Posso stabilire una corrispondenza **arbitraria** tra sequenze di {0,1} e i numeri interi. L'importante è che la corrispondenza sia univoca.

000100	0
000001	1
000010	2
000011	3
001011	4
000101	5
000110	6
000111	7
001000	8
001001	9
001010	10
000000	11
…	…

Rappresentazione dei numeri interi

Esistono però corrispondenze ‘migliori’(*) di altre. In seguito sarà chiaro perché utilizzeremo questa corrispondenza e non la precedente.

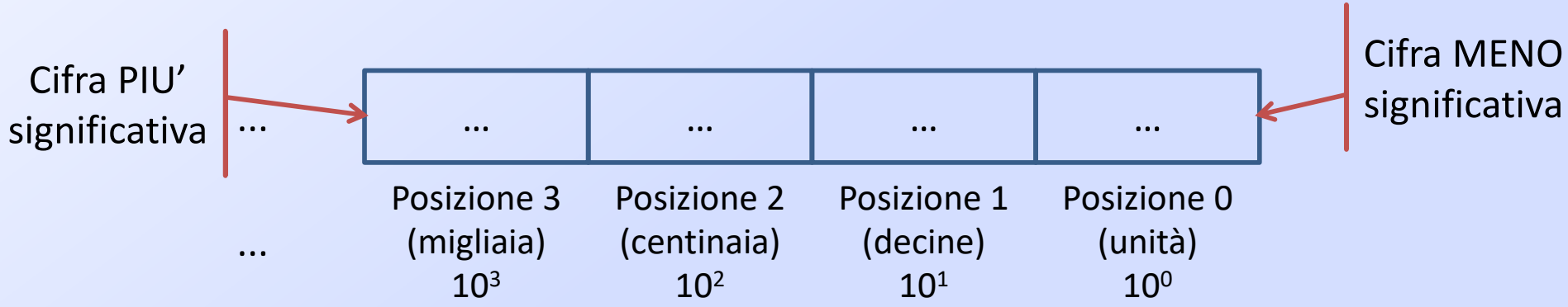
000000	0
000001	1
000010	2
000011	3
000100	4
000101	5
000110	6
000111	7
001000	8
001001	9
001010	10
001011	11
…	…

(*) esistono altre corrispondenze utili, es https://it.wikipedia.org/wiki/Codice_di_Hamming

Rappresentazione dei numeri interi

Sistema di **numerazione posizionale in base 10**
(sistema numerico decimale)

- Alfabeto di simboli (cifre) {0,1,2,3,4,5,6,7,8,9}
- **Sistema posizionale**: ogni cifra assume un **significato diverso** (peso) a seconda della **posizione** in cui si trova (unità, decine, centinaia, ecc)



Ho 10 simboli e il 'peso' associato a ciascuna posizione è dato dalle potenze di 10: 10^0 per la prima posizione, 10^1 per la seconda, 10^2 per la terza e così via

Rappresentazione dei numeri interi

Se chiamiamo a_i la cifra in *i*-esima posizione, l'insieme di simboli

a_3	a_2	a_1	a_0
10^3	10^2	10^1	10^0

rappresenta il numero

$$a_3 \cdot 10^3 + a_2 \cdot 10^2 + a_1 \cdot 10^1 + a_0 \cdot 10^0 =$$

$$= a_3 \cdot 1000 + a_2 \cdot 100 + a_1 \cdot 10 + a_0 \cdot 1$$

Esempio

Consideriamo l'insieme di simboli **1971**

$$\mathbf{1} \cdot 10^3 + \mathbf{9} \cdot 10^2 + \mathbf{7} \cdot 10^1 + \mathbf{1} \cdot 10^0 =$$

$$= \mathbf{1} \cdot 1000 + \mathbf{9} \cdot 100 + \mathbf{7} \cdot 10 + \mathbf{1} \cdot 1 = \mathbf{1971}$$

Rappresentazione dei numeri interi

Numerazione romana: sistema additivo *non posizionale*

M=1000
C=100
L=50
X=10
V=5
I=1

ciascun simbolo rappresenta sempre uno specifico valore

1971 = MCMLXXI

1971 =	M	C M	L	X	X	I
	1000	900	50	10	10	1

Rappresentazione dei numeri interi

Esempio: vantaggio della numerazione posizionale

1971 +	MCMLXXI +
1970 =	MCMLXX =
---	---
3941	. . . . .

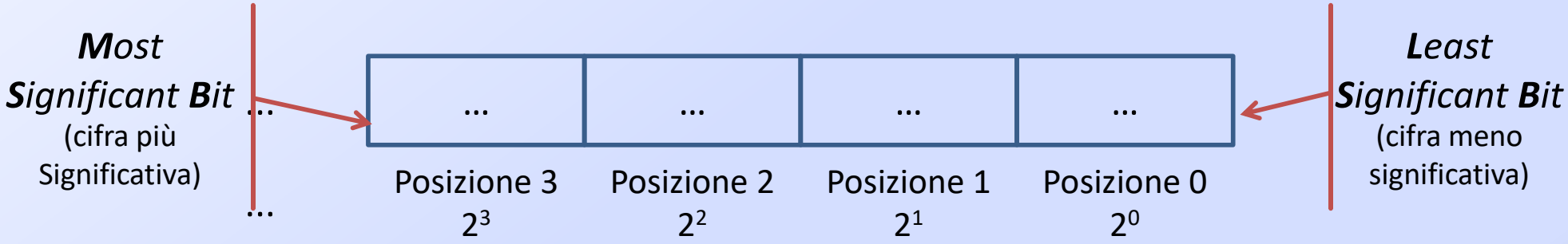
In un sistema **posizionale** a posizioni uguali corrispondono 'pesi' uguali delle cifre. Quindi utilizzando un sistema posizionale alcune operazioni di calcolo sono molto semplici grazie alla disposizione in colonna di cifre dello stesso peso. Provate a svolgere l'addizione utilizzando un sistema non posizionale, per esempio con i numeri romani...

Rappresentazione dei numeri interi

È possibile memorizzare l'informazione utilizzando solo i simboli 0 e 1?

Sistema di **<u>numerazione posizionale in base 2</u>**

- Alfabeto di simboli (**cifre binarie**) {0,1}
- Peso di ciascun simbolo dipendente dalla posizione



Ho 2 simboli e il 'peso' associato a ciascuna posizione è dato dalle potenze di 2: 2^0 per la prima posizione, 2^1 per la seconda, 2^2 per la terza e così via

(ecco da dove viene la tabella di corrispondenza vista prima)

Rappresentazione dei numeri interi

Se chiamiamo a_i la cifra in *i*-esima posizione, l'insieme di simboli $(a_3a_2a_1a_0)_2$ rappresenta il numero

a_3	a_2	a_1	a_0
2^3	2^2	2^1	2^0

$$= a_3 \cdot 2^3 + a_2 \cdot 2^2 + a_1 \cdot 2^1 + a_0 \cdot 2^0 =$$

$$= a_3 \cdot 8 + a_2 \cdot 4 + a_1 \cdot 2 + a_0 \cdot 1$$

Esempio di *cambiamento di base*

Consideriamo l'insieme di simboli (**1 1 0 0 1** $)_2$

$$\underline{\mathbf{1}} \cdot 2^4 + \underline{\mathbf{1}} \cdot 2^3 + \underline{\mathbf{0}} \cdot 2^2 + \underline{\mathbf{0}} \cdot 2^1 + \underline{\mathbf{1}} \cdot 2^0 =$$

$$= \mathbf{1} \cdot 16 + \mathbf{1} \cdot 8 + \mathbf{0} \cdot 4 + \mathbf{0} \cdot 2 + \mathbf{1} \cdot 1 = \mathbf{25}$$

$$(\mathbf{11001})_2 = (\mathbf{25})_{10}$$

Rappresentazione dei numeri interi

Sistema di numerazione posizionale

Base 10

- Alfabeto di simboli {0,1,2,3,4,5,6,7,8,9}
- Peso associato alla posizione:

  … 10^4, 10^3, 10^2, 10^1, 10^0

Base 2

- Alfabeto di simboli {0,1}
- Peso associato alla posizione:

  … 2^4, 2^3, 2^2, 2^1, 2^0

- Abbiamo espresso i numeri *interi positivi* nella loro rappresentazione binaria naturale
- La cifra che può assumere solo due valori, **0** o **1**, è chiamata **bit** (binary digit)

Rappresentazione dei numeri interi

Conversione da base 10 a base 2 : metodo delle divisioni successive.

- Si divide per 2 il numero in base 10 e si prende nota del resto della divisione
- Si applica lo stesso procedimento al risultato della divisione
- Stop quando il risultato è zero
- I resti della divisione, **letti dall'ultimo al primo**, formano il numero espresso in base 2

Rappresentazione dei numeri interi

Conversione da base 10 a base 2: esempio

$(29)_{10} = (?)_2$

29/2 = 14; R=1

14/2 = 7; R=0

7/2 = 3; R=1

3/2 = 1; R=1

1/2 = 0; R=1

$(29)_{10} = (10111)_2$

Verifica

$(11101)_2 =$

$= 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 =$

$= 16+8+4+1 = (29)_{10}$

Rappresentazione dei numeri interi

Considerazioni:

- Abbiamo esaminato il meccanismo per la conversione da base 2 a base 10 e viceversa
- È possibile rappresentare nel sistema binario anche numeri negativi o con parte decimale; occorre notare che in **generale la rappresentazione dei reali(*) è *approssimata -> ES:*** 1-1/3-1/3-1/3 ***in python***
- È possibile effettuare le usuali operazioni aritmetiche utilizzando la rappresentazione in base 2

(*)La rappresentazione (approssimata) dei reali non rientra tra gli obiettivi del corso. L'importante è ricordare che è una rappresentazione approssimata, e quindi **può introdurre errori**. Chi vuole farsi un'idea può cercare "rappresentazione in virgola mobile" oppure leggere questo: David Goldberg, What Every Computer Scientist Should Know About Floating-Point Arithmetic https://www.itu.dk/~sestoft/bachelor/IEEE754_article.pdf

Rappresentazione dei numeri interi

- Sistema di numerazione in base *b*
  - Numero naturale *b*, detto *radice* o *base*
  - *b* simboli, detti *cifre*, corrispondenti ai primi *b* numeri naturali, da 0 a *b*-1
- Posso esprimere un numero intero in base *b* mediante la sequenza

  $(a_n a_{n-1} a_{n-2} \dots a_2 a_1 a_0)_b$

  la corrispondente rappresentazione in base 10 è

  $a_n \cdot b^n + a_{n-1} \cdot b^{n-1} + a_{n-2} \cdot b^{n-2} + \dots + a_2 \cdot b^2 + a_1 \cdot b^1 + a_0 \cdot b^0 =$

  $= \sum_{i=0}^{n} a_i b^i$
- Di particolare importanza sono la numerazione **ottale** (*b*=8) e la numerazione **esadecimale** (*b*=16)

Rappresentazione dei numeri interi

ottale (*b*=8)

Alfabeto di simboli {0,1,2,3,4,5,6,7}

$(1137)_8 = 1 \cdot 8^3 + 1 \cdot 8^2 + 3 \cdot 8^1 + 7 \cdot 8^0 = 512+64+24+7 = (607)_{10}$

607/8 = 75; R=7

75/8 = 9; R=3

9/8 = 1; R=1

1/8 = 0; R=1

$(607)_{10} = (1137)_8$

Rappresentazione dei numeri interi

La conversione da base 8 a base 2 è molto semplice. Le singole cifre 0... 7 in base 8 hanno lo stesso valore che avrebbero in base 10. Convertiamo separatamente ogni cifra ottale in **3 bit** (semplice vedere che 3 bit sono necessari e sufficienti)

$$(63)_8 = (\underbrace{110}_{6}\ \underbrace{011}_{3})_2$$

Viceversa per passare da base 2 a base 8 raggruppiamo 3 cifre alla volta

$$(\underbrace{110}_{6}\ \underbrace{011}_{3})_2 = (63)_8$$

Rappresentazione dei numeri interi

esadecimale (HEX) (*b*=16)

Alfabeto di simboli {0,1,2,3,4,5,6,7, 8, 9, **A, B, C, D, E, F**}

(**A** vale 10, ... **F** vale 15)

$(11FE)_{16} = 1 \cdot 16^3 + 1 \cdot 16^2 + F \cdot 16^1 + E \cdot 16^0 = 4096+256+240+14=$

$=(4606)_{10}$

4606/16 = 287; R=14 (E)

287 /16 = 17; R=15 (F)

17/16 = 1; R=1

1/16 = 0; R=1

$(4606)_{10} = (11FE)_{16}$

Rappresentazione dei numeri interi

La conversione da base 16 a base 2 segue lo schema visto per la numerazione in base 8 ma la corrispondenza è tra cifra esadecimale e gruppo di **4 bit** (perché?)

$$(3A7)_{16} = (\underbrace{0011}_{3}\ \underbrace{1010}_{A}\ \underbrace{0111}_{7})_2$$

$$(\underbrace{0011}_{3}\ \underbrace{1010}_{A}\ \underbrace{0111}_{7})_2 = (3A7)_{16}$$

Conversione da base 8 a base 16 e viceversa: la cosa più rapida è passare attraverso la base 2.

Rappresentazione dei numeri interi

Fissato il numero di bit a disposizione, quale è il numero più grande che posso esprimere? O anche: quale è il numero di 'configurazioni' differenti che posso rappresentare?

Esempio in base 10

Cifre a disposizione	Massimo valore rappresentabile	Valori rappresentabili	Differenti configurazioni rappresentabili
1	9=10-1	da 0 a 9	10
2	$99=10^2-1$	da 0 a 99	$10^2=100$
3	$999=10^3-1$	da 0 a 999	$10^3=1000$
4	$9999=10^4-1$	da 0 a 9999	$10^4=10000$

Rappresentazione dei numeri interi

Esempio in base 2

Cifre (bit) a disposizione	Massimo valore rappresentabile		Valori rappresentabili		Differenti configurazioni rappresentabili
	in binario	in decimale	in binario	in decimale	
1	1	$1=2^1-1$	da 0 a 1	da 0 a 1	2
2	11	$3=2^2-1$	da 0 a 11	da 0 a 3	$2^2=4$
3	111	$7=2^3-1$	da 0 a 111	da 0 a 7	$3^3=8$
4	1111	$15=2^4-1$	da 0 a 1111	da 0 a 15	$2^4=16$
…	…	…	…	…	…
8	11111111	$255=2^8-1$	da 0 a 11111111	da 0 a 255	$2^8=256$

Rappresentazione dei numeri interi

Regola generale:

- con N cifre decimali posso esprimere 10^N differenti 'configurazioni'
  Il più grande numero rappresentabile è 10^N -1
- con N cifre binarie (bit) posso esprimere 2^N differenti 'configurazioni'
  Il più grande numero rappresentabile è 2^N -1
  - Con **un bit** posso esprimere due possibili 'configurazioni'
  - Con **8 bit** posso esprimere 2^8=256 possibili 'configurazioni'

Rappresentazione dei numeri interi

È facile rispondere alla domanda opposta: quanti bit occorrono per rappresentare (in base 2) il numero *m*?

Occorrono N bit, con $2^N - 1 \geq m$

Esempio: se voglio rappresentare il numero 10 occorrono 4 bit, poiché $2^4 - 1 \geq 10$. Infatti $(10)_{10} = (1010)_2$

Tre bit non sarebbero sufficienti: il numero più elevato esprimibile con 3 bit è $(111)_2 = (7)_{10}$

Rappresentazione dei numeri interi

RIASSUMENDO:

- I sistemi di elaborazione presi in esame sono costituiti da elementi che possono assumere i valori **0** o **1**
- Ciascuno di questi elementi può memorizzare **un bit**, che può essere utilizzato nella rappresentazione degli interi.

Non vedremo nel dettaglio come il computer può rappresentare altri enti numerici (numeri negativi, frazionari, ecc)

Il byte e i suoi multipli

Un **byte** è un insieme di **8 bit**. Con un byte è possibile rappresentare $2^8 = 256$ possibili valori distinti.

I multipli del byte, riportati in tabella, sono definiti per mezzo delle potenze di 2. Tuttavia negli usi pratici (non specialistici) si possono approssimare i valori esatti agli usuali valori dei multipli delle unità di misura.

Nome	Simbolo	Valore esatto	Valore approssimato
kilobyte	KB	2^{10}= 1024 byte	10^3=1000 byte
megabyte	MB	2^{20}= 1024*1024 byte =1'048'576 byte	10^6=1 milione di byte
gigabyte	GB	2^{30}= 1024*1024*1024 byte	10^9=1 miliardo di byte
terabyte	TB	2^{40}= 1024*1024*1024*1024 byte	10^{12}=1000 miliardi di byte
…			

Il byte e i suoi multipli

Esempio:

12 500 byte equivalgono a *circa* 12.5 KB

25 700 000 byte equivalgono a *circa* 25.7 MB

32 200 MB equivalgono a *circa* 32.2 GB

(NB il separatore decimale utilizzato è il *punto*, non la *virgola*, secondo la notazione anglosassone)

Nome	Simbolo	Valore esatto	Valore approssimato
kilobyte	KB	2^{10}= 1024 byte	10^3=1000 byte
megabyte	MB	2^{20}= 1024*1024 byte =1'048'576 byte	10^6=1 milione di byte
gigabyte	GB	2^{30}= 1024*1024*1024 byte	10^9=1 miliardo di byte
terabyte	TB	2^{40}= 1024*1024*1024*1024 byte	10^{12}=1000 miliardi di byte
…			

Rappresentazione dei caratteri

È possibile memorizzare caratteri utilizzando elementi bistabili?

Abbiamo visto che è possibile memorizzare i numeri interi. A questo punto è sufficiente stabilire una *corrispondenza* tra i valori numerici e caratteri.

Esempio

Carattere da rappresentare	Valore numerico associato *arbitrariamente*	Rappresentazione binaria
A	0	000
B	1	001
C	2	010
D	3	011
…	…	…

Rappresentazione dei caratteri

Quanti bit occorrono per poter codificare un carattere?

Con 5 bit posso codificare $2^5=32$ configurazioni possibili, quindi posso pensare di usare 5 bit per codificare l'alfabeto

carattere	valore numerico	rappresentazione binaria
a	0	0 0 0 0 0
b	1	0 0 0 0 1
c	2	0 0 0 1 0
d	3	0 0 0 1 1
…	…	…
w	22	1 0 1 1 0
x	23	1 0 1 1 1
y	24	1 1 0 0 0
z	25	1 1 0 0 1

Rappresentazione dei caratteri

Vi invito a riflettere sull'esempio precedente.

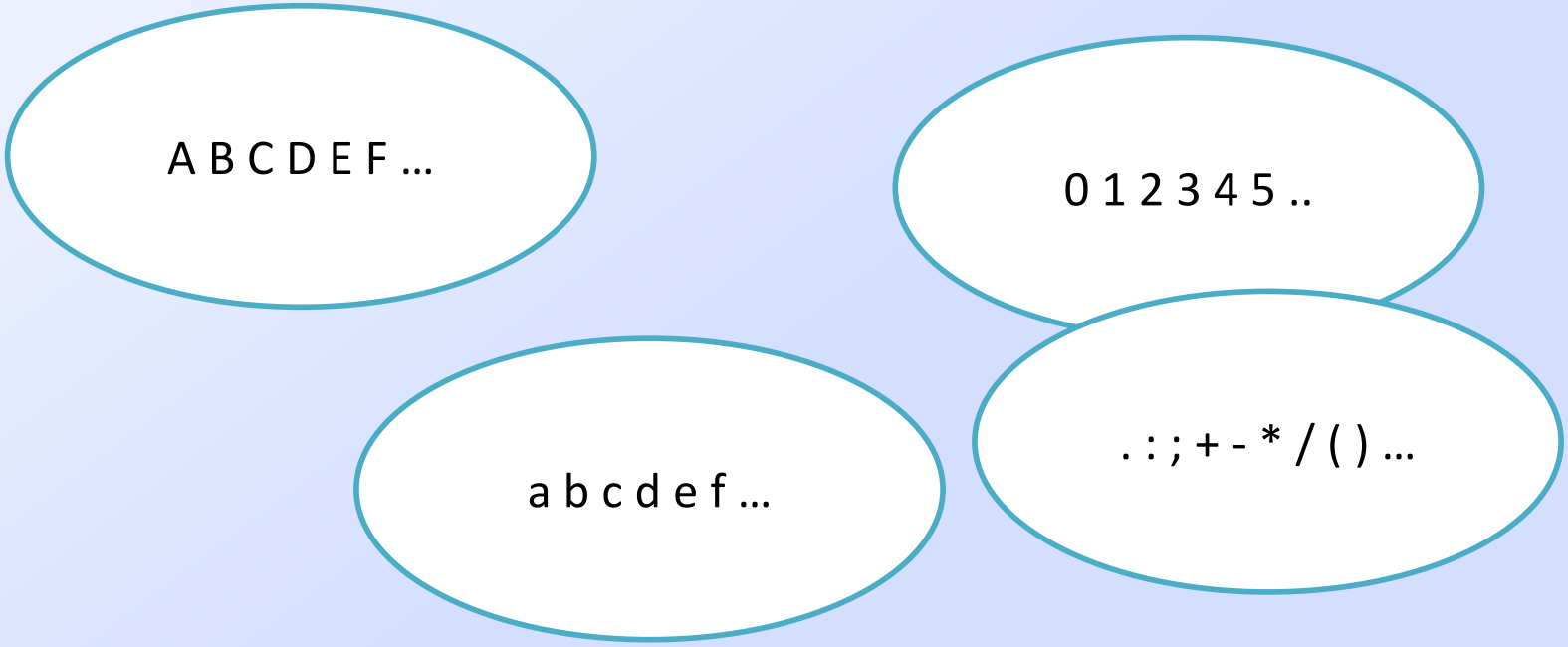
Se devo identificare **un** preciso carattere fra i 26 possibili, 5 bit sono sufficienti. Una volta stabilita la tabella di corrispondenza, il valore assunto dai 5 bit (per esempio: **00011**) identifica il carattere (per esempio: **d**).

Lo stesso tipo di ragionamento si applica a molti altri casi che vedremo, quindi **assicuratevi che sia ben chiaro.**

carattere	valore numerico	rappresentazione binaria
a	0	0 0 0 0 0
b	1	0 0 0 0 1
c	2	0 0 0 1 0
d	3	0 0 0 1 1
…	…	…
w	22	1 0 1 1 0
x	23	1 0 1 1 1
y	24	1 1 0 0 0
z	25	1 1 0 0 1

Rappresentazione dei caratteri

- 26 caratteri sono sufficienti?



Rappresentazione dei caratteri

- Il **<u>codice ASCII a 7 bit</u>** (*) è un sistema di codifica che rappresenta i 52 caratteri alfabetici (maiuscole e minuscole), le 10 cifre, i 32 simboli **!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~** ed altri caratteri speciali per un totale di **128 caratteri**.
- Per codificare 128 caratteri occorrono 7 bit, infatti con 7 bit ho 2^7=128 configurazioni diverse, da 0 a 2^7-1=127
- Il codice ASCII è uno standard, tuttavia i caratteri elencati **non sono sufficienti**: mancano per esempio alcuni caratteri utilizzati in varie lingue: **è è ò ì ù ö ñ** …

(*) American Standard Code for Information Interchange, ovvero Codice Standard Americano per lo Scambio di Informazioni

Rappresentazione dei caratteri

- Il **codice ASCII Esteso** (*) utilizza 8 bit; può quindi rappresentare fino a **256 caratteri**.
- È uno 'standard de facto'
- Utilizzando il codice ASCII esteso con un byte (8 bit) è possibile rappresentare un carattere
- Il codice ASCII esteso rappresenta, oltre ai caratteri presenti nell'ASCII a 7 bit,
  - lettere accentate di vari alfabeti
  - simboli monetari e matematici
  - caratteri grafici
  - lettere dell'alfabeto greco.
- La sequenza di tasti **ALT+ codice ASCII** richiama il carattere specificato

(*) esistono versioni leggermente differenti dell'ASCII esteso. Citiamo ad esempio la versione 'Latin-1', più formalmente ISO/IEC 8859-1, riportata in http://en.wikipedia.org/wiki/ISO_8859-1

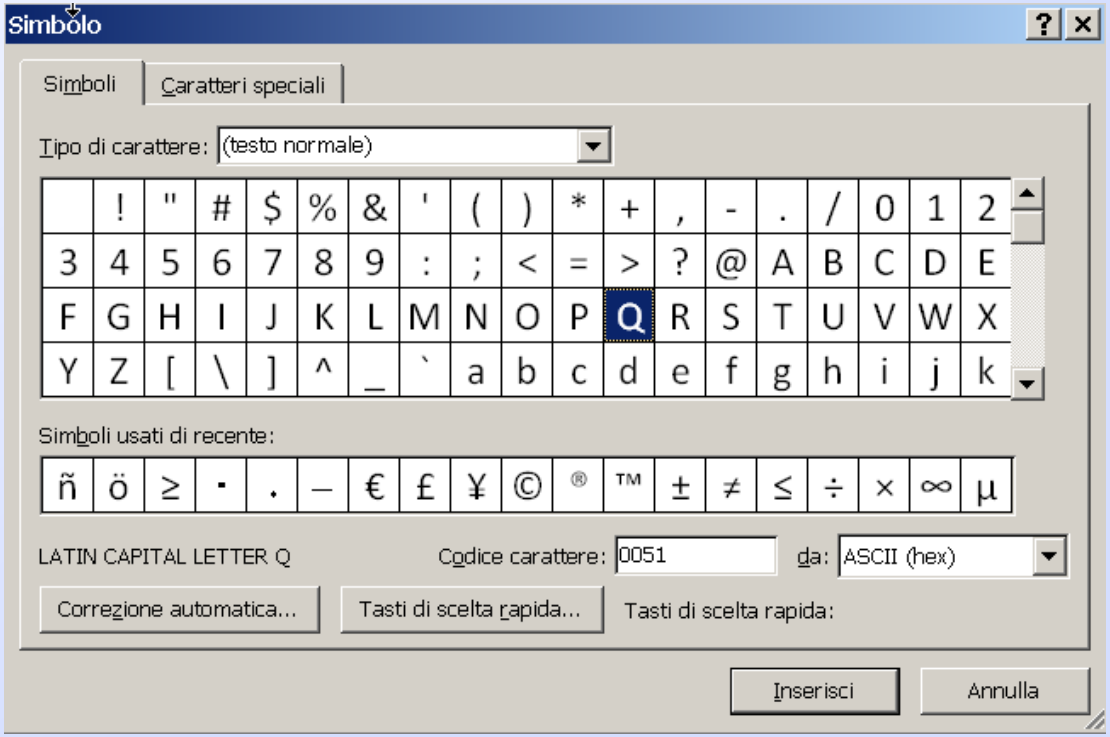
Rappresentazione dei caratteri

- Anche 256 caratteri sono insufficienti – mancano infatti i caratteri utilizzati negli alfabeti cirillico, arabo, ecc
- **UNICODE**(*) è un sistema di codifica ideato per risolvere questo ed altri problemi
- UNICODE si basa su una codifica a 8, 16 o 32 bit, e assegna un numero univoco a ogni carattere,
  - indipendentemente dalla piattaforma
  - indipendentemente dall'applicazione
  - indipendentemente dalla lingua
- 8 bit -> 256 caratteri
- 16 bit -> 2^{16}=65 536 caratteri
- 32 bit -> 2^{32}=4.29 miliardi di caratteri

(*) http://www.unicode.org/standard/translations/italian.html

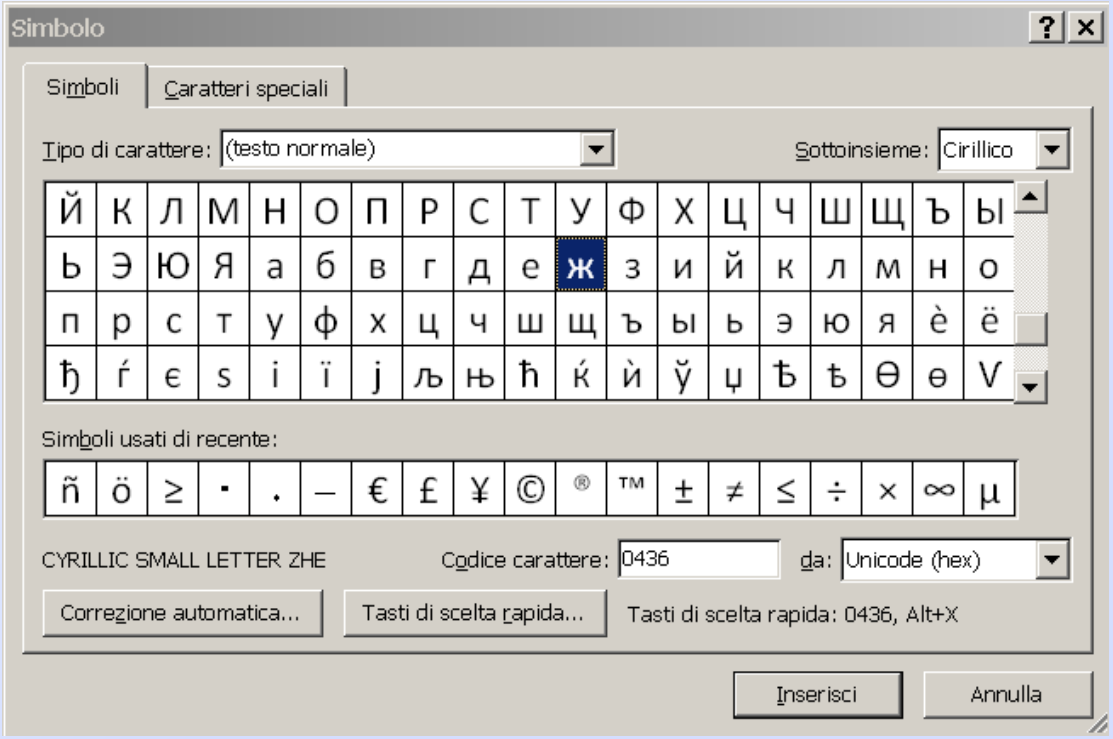
Rappresentazione dei caratteri

- Esempio inserimento caratteri ASCII (Windows)



Rappresentazione dei caratteri

- Esempio inserimento caratteri UNICODE (Windows)



Rappresentazione dei caratteri

- Con un byte (8 bit) è possibile rappresentare un carattere (*)
- Quanti byte occorrono per memorizzare
  - Una pagina di testo?
  - Una tesi di laurea?
  - Una foto?
  - Un brano musicale?
  - Un filmato?
  - …

(*) Utilizzando il codice ASCII esteso

Rappresentazione dei caratteri

- Quanti byte occorrono per memorizzare una pagina di testo?

•Calcolo *approssimato.* Il numero di caratteri presente in una pagina dipende dall'interlinea, dalla dimensione dei caratteri, ecc

•Notare che *anche gli spazi occupano 1 byte*

•Un foglio A4 contiene circa 80*28=2240 caratteri, circa 2.2KB

- Quanti byte occorrono per memorizzare una tesi di laurea?
  - Considerando una tesi composta di <u>solo testo</u>, è sufficiente moltiplicare il numero delle pagine per l'occupazione della singola pagina
  - Vedremo come valutare anche l'occupazione di memoria di <u>immagini, video, audio</u>, ecc
  - Vedremo come l'occupazione di memoria incide sulla gestione dell'informazione (per es: quali supporti utilizzare per memorizzarla?)

ESERCIZI

- Data una serie di numeri in base 10, convertirli in base 2. Verificare la correttezza della conversione riconvertendoli in base 10
- Convertire da base 8 a base 10 e viceversa e da base 8 a base 2 e viceversa
- Convertire da base 16 a base 10 e viceversa e da base 16 a base 2 e viceversa
- Convertire da base 8 a base 16 e viceversa (direttamente, passando per base 2)
- Date n cifre in base b, quale è il massimo numero rappresentabile?
- Quanti bit occorrono per rappresentare il numero $(n)_{10}$ in base 2?

Cosa abbiamo visto

- L'informazione
- Rappresentazione dei numeri interi
- Come passare dalla rappresentazione in base b1 alla rappresentazione in base b2
- Quanti bit occorrono per rappresentare un certo numero
- Il byte e i suoi multipli
- Rappresentazione dei caratteri

Rappresentazione delle informazioni

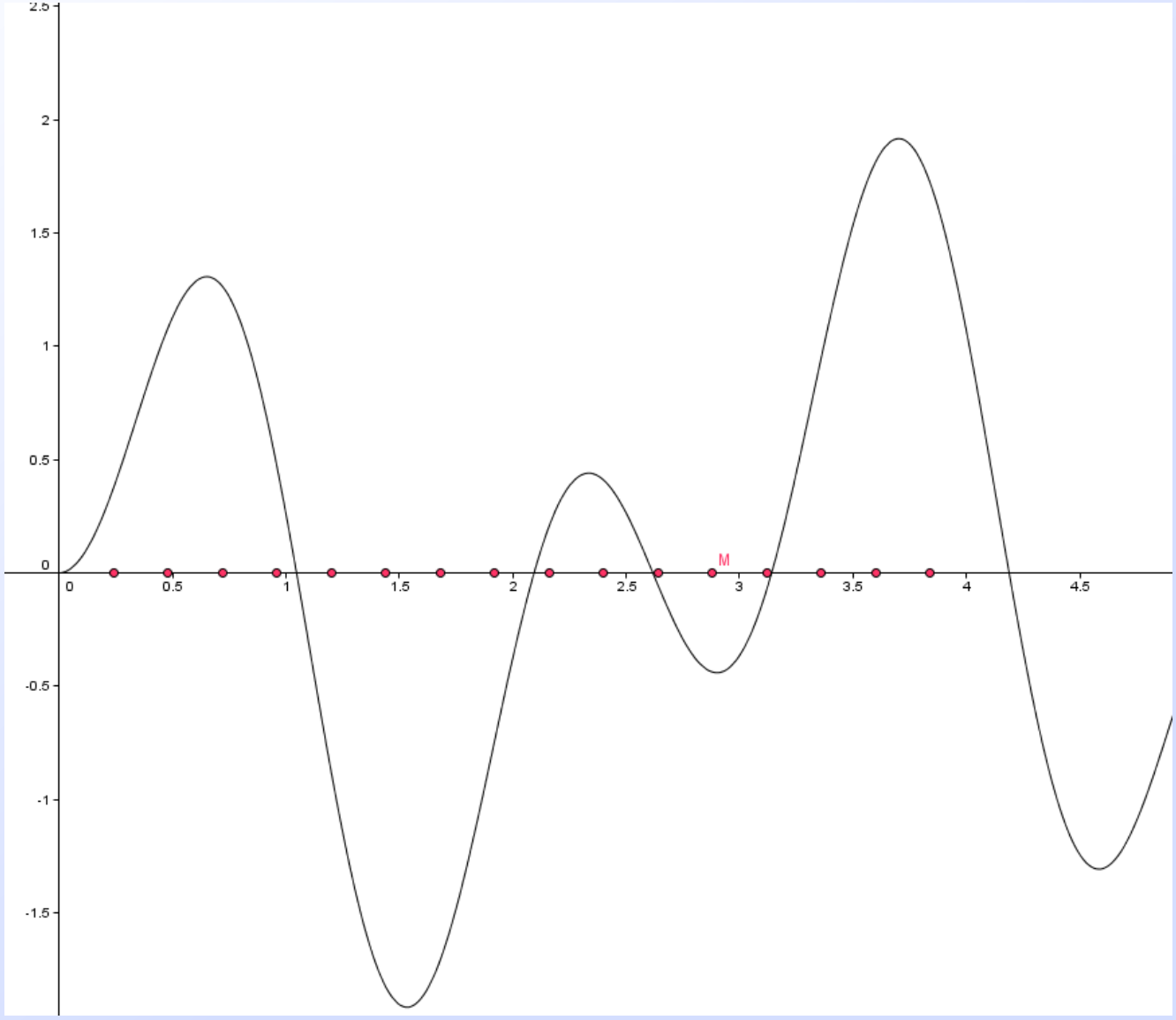
Seguono cenni ad argomenti che non possono essere totalmente ignorati da un ingegnere ma che non svilupperemo oltre

Rappresentazione delle informazioni

Problema: trasformare un segnale da **analogico** a **digitale**

- Segnale **analogico**: rappresentazione di una grandezza mediante una grandezza *analoga.* È un segnale continuo che può assumere una gamma infinita di valori
- Segnale **digitale**: rappresentazione di una grandezza mediante un insieme di *valori numerici*.

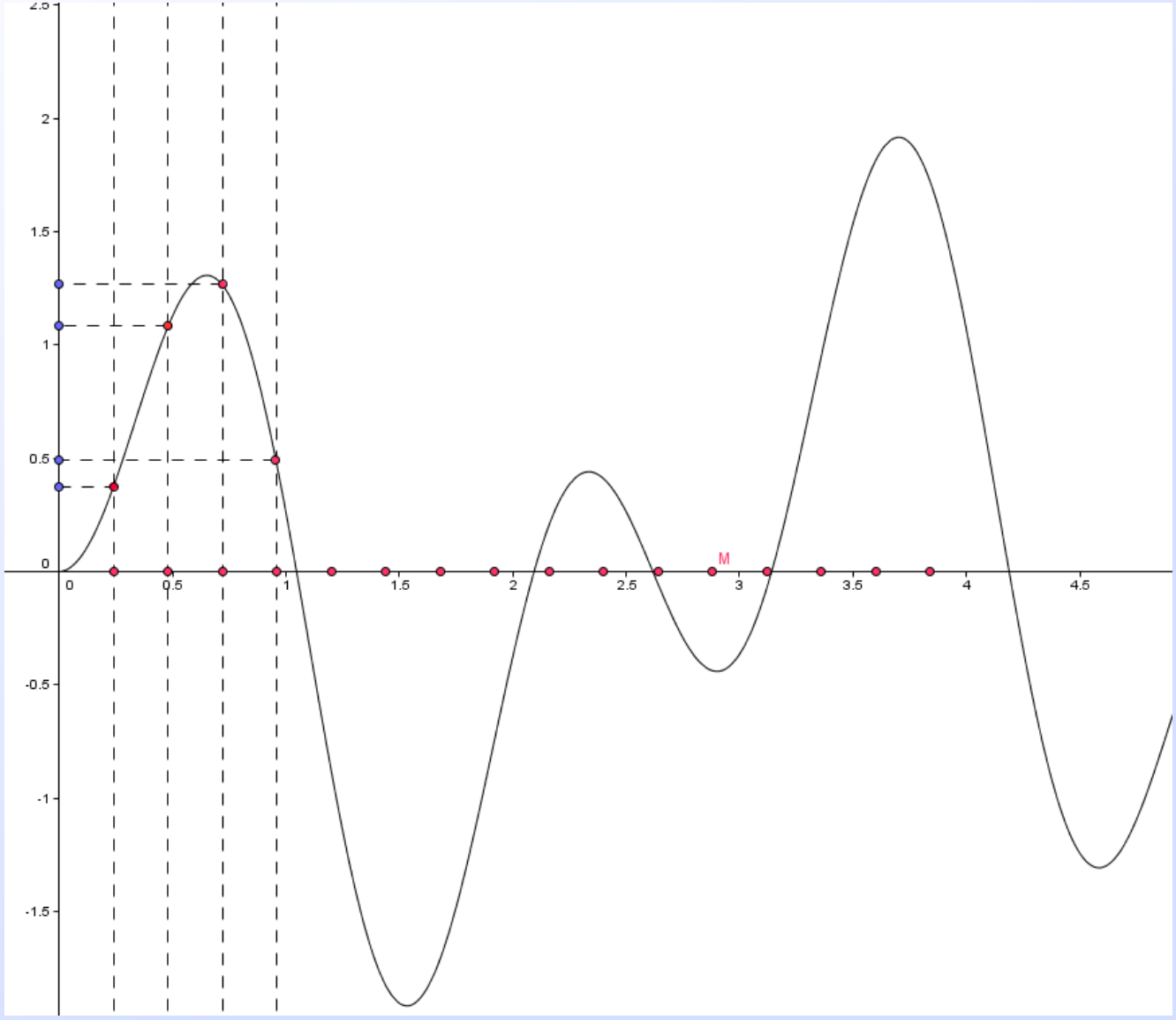
Campionamento e quantizzazione



Esempio di grandezza analogica

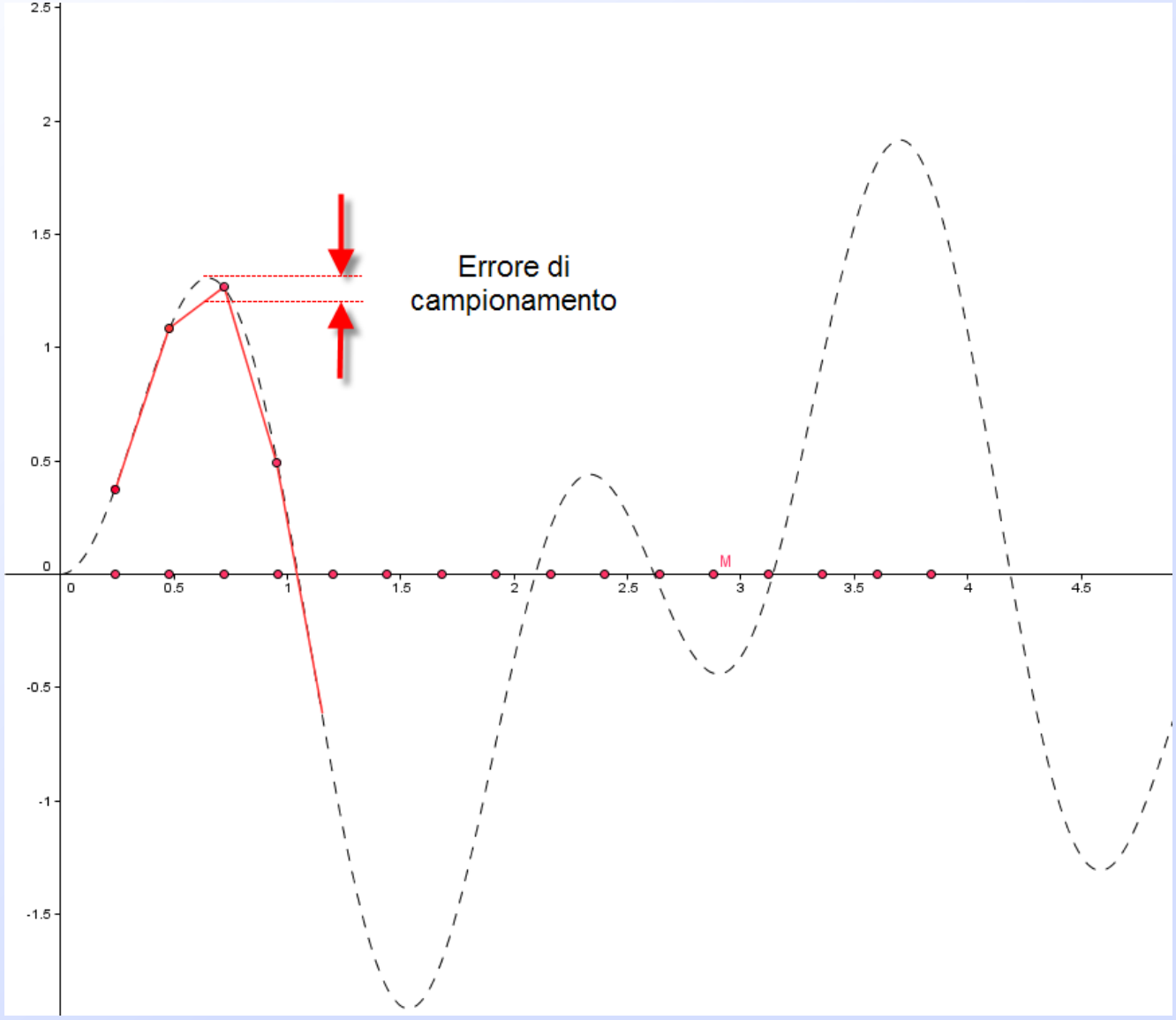
Si effettua un campionamento ad intervalli di tempo prefissati

Campionamento e quantizzazione



Otteniamo una serie di misure che rappresentano il valore della grandezza in esame negli istanti di tempo considerati

Campionamento e quantizzazione

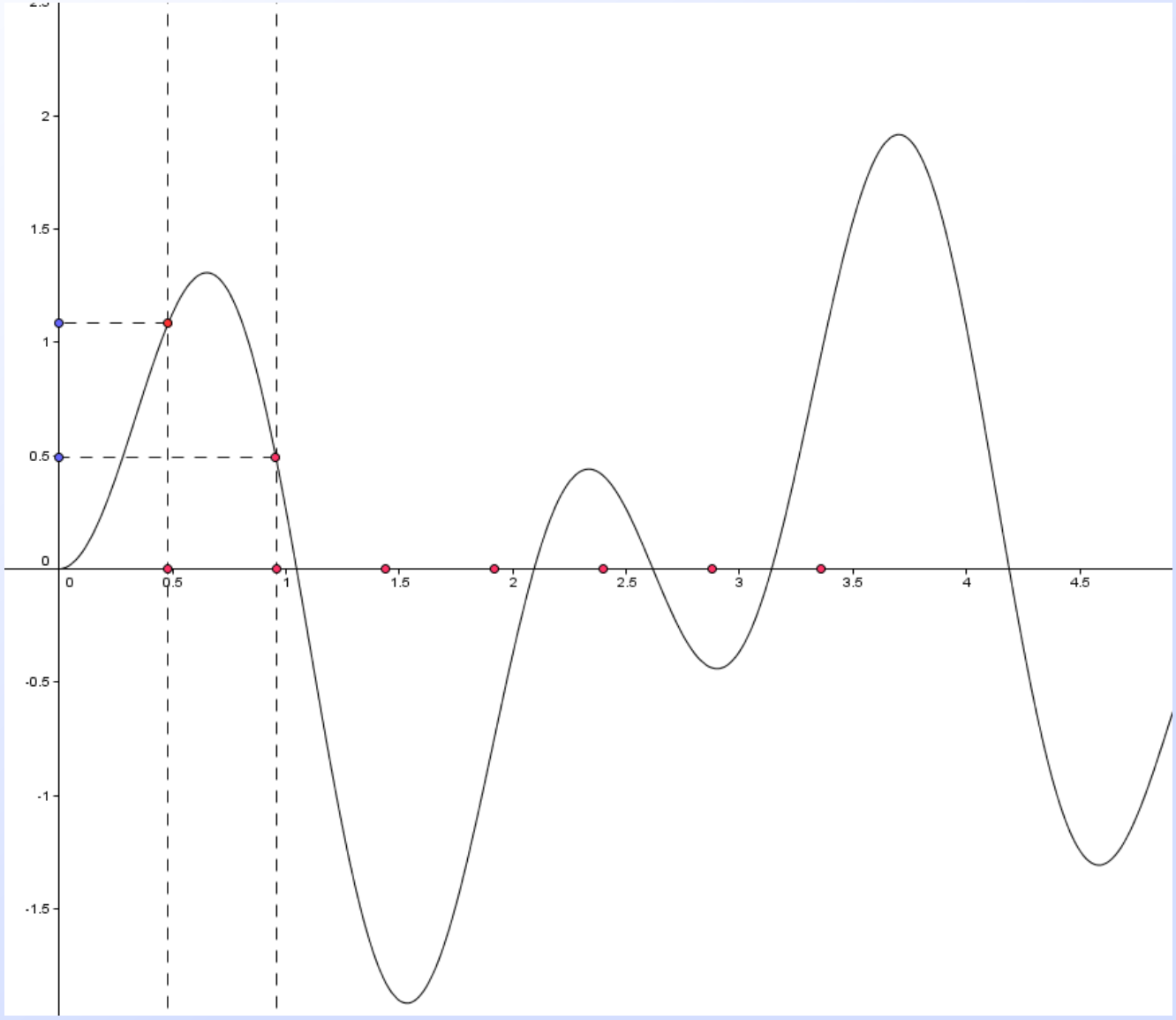


L'obiettivo è ricostruire l'informazione originale mediante le misure ottenute.

Anche ipotizzando di ottenere *misure perfette* si commette un errore dovuto al fatto che il segnale analogico è costituito da un insieme infinito di valori, mentre il processo di campionamento considera un insieme finito di misure.

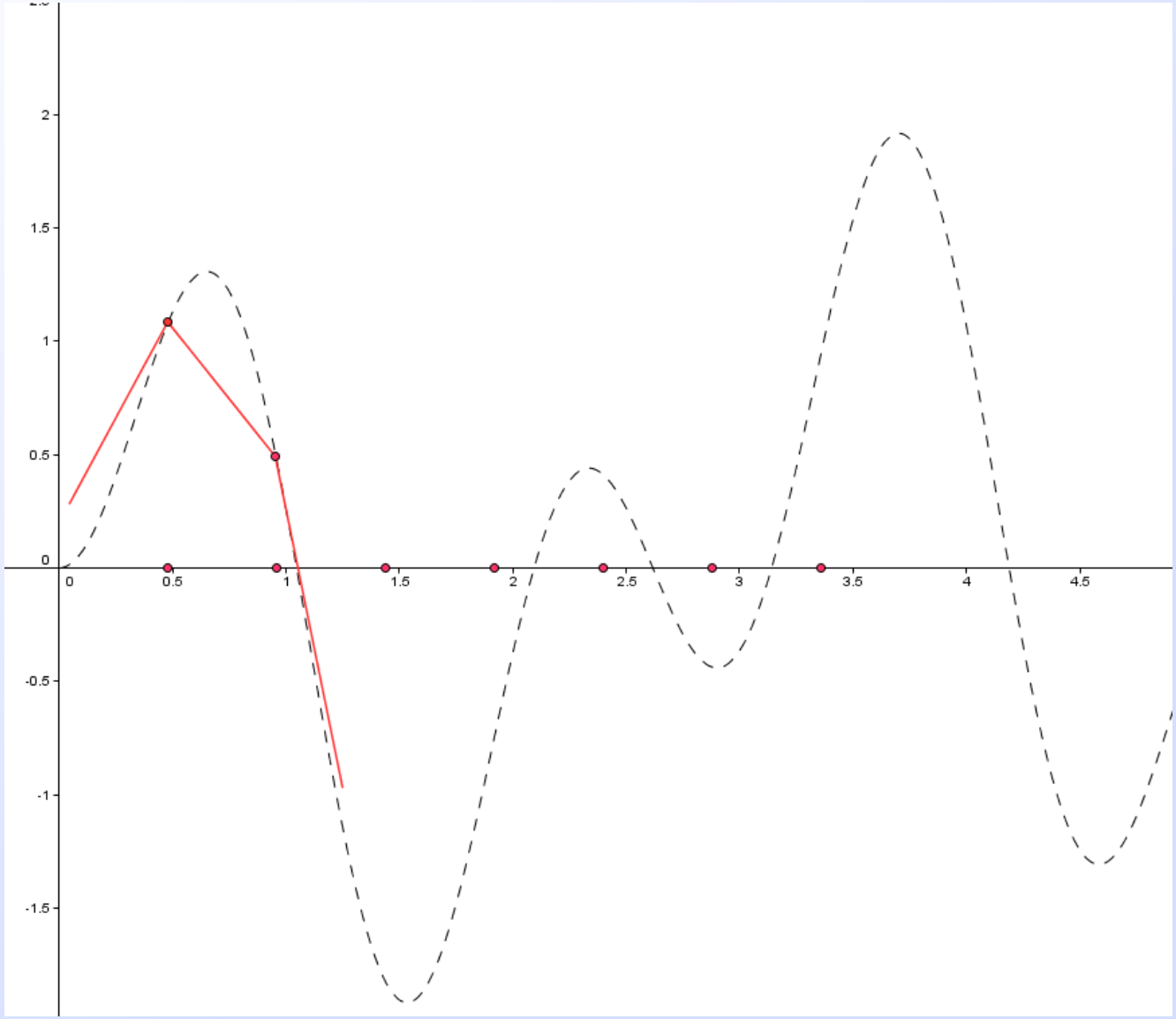
La spezzata in rosso rappresenta il tentativo di ricostruire il segnale originale a partire dalle misure.
Si nota che nell'istante della misura la differenza tra segnale vero e segnale ricostruito è nulla, mentre negli altri punti commettiamo un errore.

Campionamento e quantizzazione



Se diradiamo ulteriormente gli atti di campionamento…

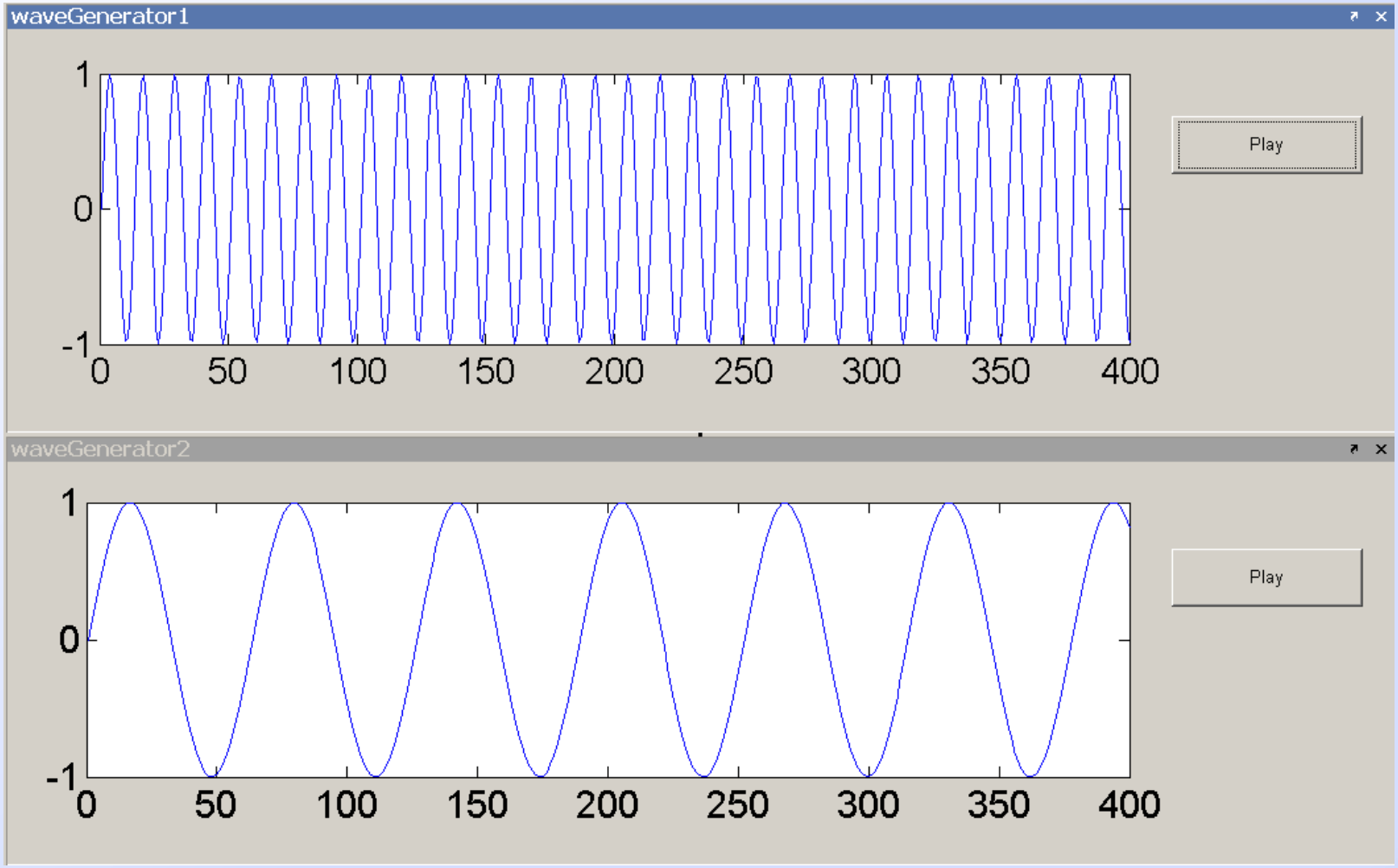
Campionamento e quantizzazione



… l'errore aumenta.

L'errore di campionamento è legato alla **frequenza di campionamento** e alla **frequenza del segnale**. Intuitivamente se il segnale analogico varia con rapidità (frequenze elevate) è necessario aumentare il numero di campioni per ricostruire il segnale. Aumentare il numero di campioni significa aumentare la frequenza di campionamento (o diminuire l'intervallo tra un campione e il successivo)

Campionamento e quantizzazione



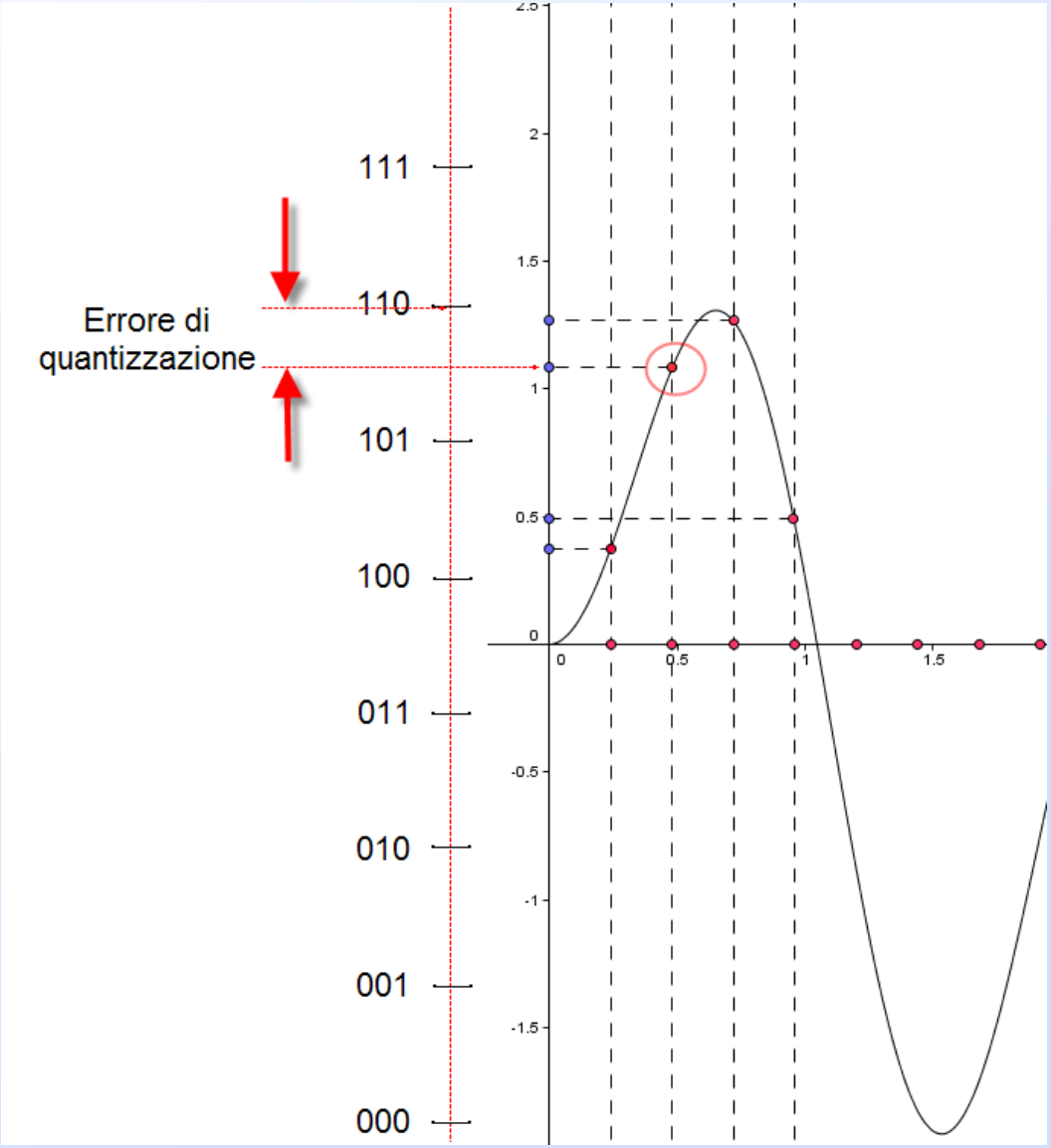
Esempio: due segnali a differente frequenza

Campionamento e quantizzazione

La frequenza di campionamento deve essere maggiore del doppio della massima frequenza (di interesse) del segnale

- Segnale vocale per telefonia: mi interessano le frequenze fino a circa 3.5 kHz, la frequenza di campionamento è di circa 7 kHz
- Audio di qualità ‘CD’: mi interessano le frequenze fino a circa 20 kHz, la frequenza di campionamento è di circa 40 kHz

Campionamento e quantizzazione



Nella realtà posso utilizzare **solo un certo numero di bit** per rappresentare il risultato del campionamento.

Tali misure quindi saranno note solo in forma approssimata. A ciascuna misura, originariamente presa da un insieme infinito di valori continui, associo un valore preso da un **insieme finito di valori discreti.**

Al valore continuo misurato inizialmente è associato un valore scelto fra un insieme fissato di possibili quantità. Ottengo quindi una misura **quantizzata** (discreta)

Nell'esempio, a ciascuna misura sono dedicati 3 bit. Ho quindi 8 possibili **livelli differenti di quantizzazione**. Il secondo valore campionato ha un valore tra il livello 101 e il livello 110. Ad esso viene associato il valore 110, con un conseguente **errore di quantizzazione**

Campionamento e quantizzazione

La conversione di un segnale analogico in segnale digitale comporta due tipi di errore: **errore di campionamento** e **errore di quantizzazione**.

- Si può ridurre l'errore di campionamento aumentando il numero di campioni (misure)
- Si può ridurre l'errore di quantizzazione aumentando il numero di bit utilizzati per definire i livelli di quantizzazione.
- In entrambi i casi questo si traduce in un aumento del numero di bit necessari a rappresentare il segnale

Campionamento e quantizzazione

ESEMPIO

- Formato standard per i CD audio
  - frequenza di **campionamento** di 44'100 Hz (*), cioè l'andamento della pressione sonora è misurato (campionato) ad intervalli regolari, esattamente 44100 volte al secondo.
  - **quantizzazione** su 65'536 livelli
    (ciascun campione viene codificato da una sequenza di 16 bit)
- Un secondo di musica stereo richiede 44'100 campioni di 16 bit (2 byte) ciascuno per due canali, quindi 176'400 byte in tutto.
- Un minuto di musica stereo richiede circa 176000 * 60 = 10 MByte di dati
- 74 minuti di musica stereo, con la codifica qui illustrata, richiedono circa 74*10MByte= 740 Mbyte, che è proprio la dimensione di un CD audio
- L'errore che si commette nella ricostruzione del segnale sonoro è difficilmente rilevabile da parte di un orecchio umano.

(*) La frequenza di campionamento scelta garantisce l'assenza di errori di campionamento sul segnale fino alla frequenza di 20000 Herz (teorema di Nyquist-Shannon, che esula dagli scopi del corso)

Codifica Immagini (cenni)

La **grafica bitmap**, o **grafica raster** (griglia), è una tecnica(*) utilizzata per rappresentare un'immagine. Un'immagine descritta con questo tipo di grafica è chiamata *immagine bitmap* o *immagine raster*.

- l'immagine viene vista come un *mosaico* e ad ogni 'tessera' del mosaico, chiamata pixel (*picture element*) viene associato uno specifico colore.
- Il colore può essere definito con due tecniche:
  - se l'immagine contiene pochi colori (massimo 256) si crea un elenco (*palette*) dei colori da utilizzare e nella scacchiera viene inserito l'indice che punta allo specifico colore del pixel;
  - Se l'immagine contiene molti più colori al singolo pixel si associa un valore numerico che definisce direttamente il colore.

(*) Questo non è l'unico modo per rappresentare un'immagine. Nella *grafica vettoriale* un'immagine è descritta mediante un insieme di primitive geometriche che descrivono punti, linee, curve e poligoni che compongono l'immagine.

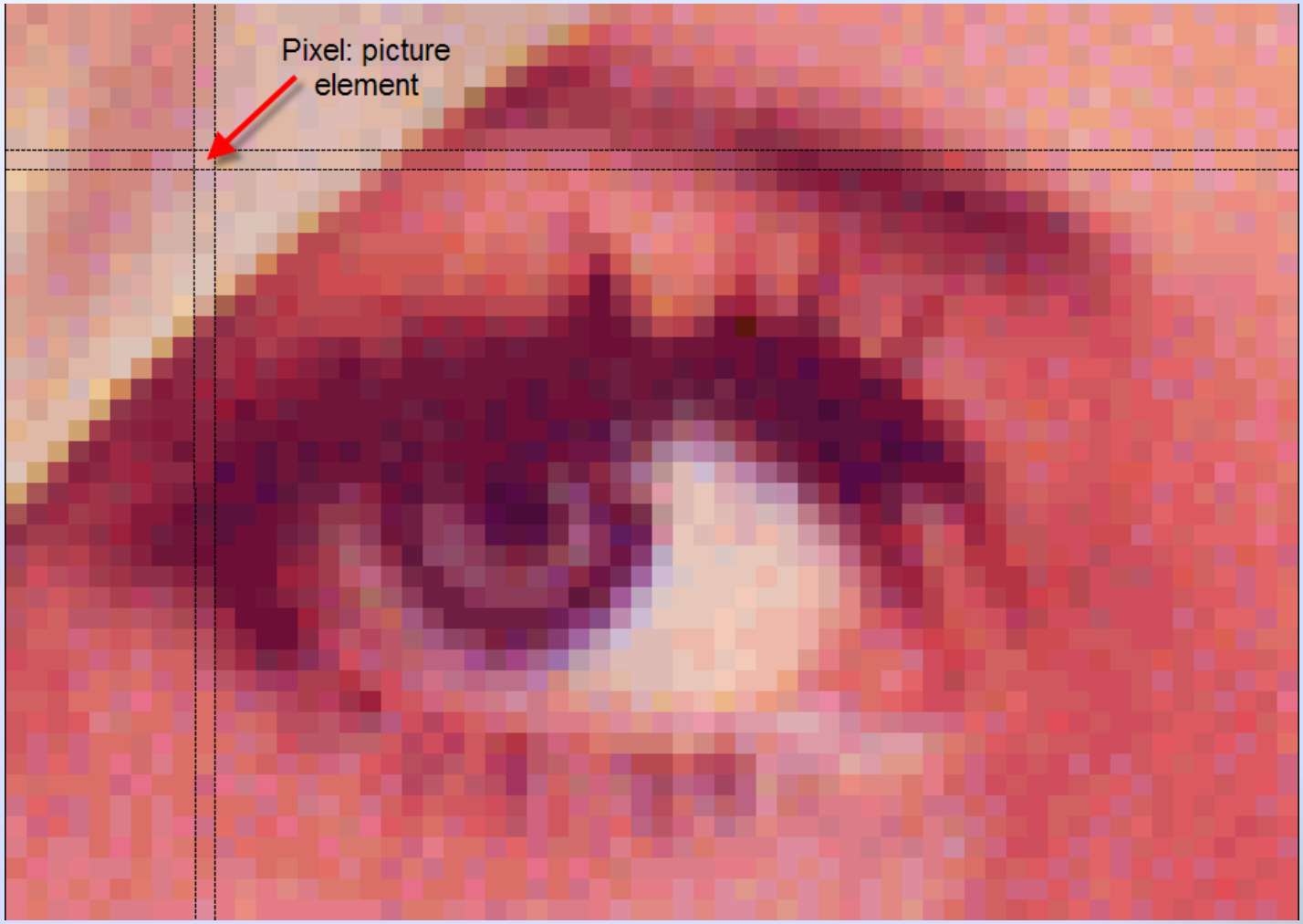
Codifica Immagini (cenni)

L’immagine rappresentata in questo modo è caratterizzata da due proprietà

- Risoluzione: è determinata dal numero di pixel contenuti nell'unità di misura considerata

- Profondità di colore: è definita dal numero di bit dedicati ad ogni pixel per descrivere il colore. Maggiore è il numero di bit, maggiore è il numero di colori che è possibile descrivere.

Codifica Immagini (cenni)



Codifica Immagini (cenni)

Nome	Estensione	Profondità (BPP)	note
Bitmap	.bmp	1, 4, 8, 16 o 24 bit, che corrispondono a 2, 16, 256, circa 65.000 e circa 16.7 milioni di colori. Con meno di 16 bit viene utilizzata una *palette* per la rappresentazione dei colori. Le immagini ad 8 bit possono utilizzare una rappresentazione a toni di grigio senza *palette*.	Inadatto alla pubblicazione di immagini su internet a causa della grande quantità di byte richiesti. Per le fotografie professionali si preferisce il formato TIFF.
Gif Graphics Interchange Format	.gif	8 bit (256 colori), con *palette*.	Utilizzato su Internet, ma solo piccole immagini o con pochi dettagli

Codifica Immagini (cenni)

Nome	Estensione	Profondità (BPP)	note
Jpeg Joint Photographic Experts Group	.jpg; .jpe	24 bit (*)	Compressione con perdita di informazione. Formato diffusissimo su internet e su macchine fotografiche. Non adatto per fotografie professionali.
TIFF Tagged Image File Format	.tif	24 bit (corrispondenti a 16.7 milioni di colori) o più. Per esempio se si utilizza lo spazio RGB ad ogni canale di colore sono associati 8 bit.	Permette di rappresentare immagini con diversi spazi di colore: scale di grigio, RGB, CMYK, ecc Sono possibili diversi metodi di compressione senza perdita di informazione. Per questi motivi è molto utilizzato nella grafica professionale.

Codifica Immagini (cenni)

- BPP: Bit per pixel, ‘profondità di colore’ dell’immagine – determina il numero di colori rappresentabili nella stessa immagine
- Il termine profondità di colore (color depth) è utilizzato nella grafica digitale per indicare la quantità di bit necessari per rappresentare il colore di un singolo pixel in un'immagine. Profondità di colore superiori consentono una gamma più vasta di tonalità distinte.

- Palette: la palette (tavolozza) è una lista di colori scelti nell'insieme di tutti quelli disponibili. La rappresentazione dei pixel di un'immagine attraverso indici nella tavolozza permette un significativo risparmio di memoria rispetto ad altre tecniche, ma limita la gamma di colori utilizzabili a quelli presenti nella tavolozza.

Codifica Immagini (cenni)

ESEMPIO: Devo rappresentare un'immagine usando solo sei colori.

- Soluzione senza palette: scelgo i sei colori una volta per tutte, **per tutte le possibili immagini** che dovrò rappresentare. Scegliendo per es bianco, nero, rosso, blu, verde, viola, dovendo rappresentare un'immagine che contiene solo sfumature di rosso, blu e viola (per es un tramonto) userò solo tre colori. Lo spazio (bit) utilizzato per rappresentare bianco, nero, verde è sprecato – quei colori non sono utilizzati. Inoltre la qualità della mia immagine sarà scadente, perché è realizzata utilizzando solo tre colori, non sei come potenzialmente avrei potuto fare.
- Soluzione con palette: scelgo i sei colori in base ad una analisi statistica dei colori presenti nell'immagine, dato che la palette è associata a quella particolare immagine. Se l'immagine contiene solo sfumature di rosso, blu e viola (per es un tramonto), la mia palette potrebbe contenere due sfumature di rosso, due di blu, due di viola ottenendo un risultato migliore rispetto al precedente.

Codifica Immagini (cenni)

- La scelta della palette può essere effettuata automaticamente dal programma di elaborazione delle immagini, ma la scelta del formato da utilizzare (con o senza palette, numero di bit, ecc) è compita dell'utente.
- Il numero di elementi in una tavolozza raramente supera i 256 colori. Quando è necessario un numero di colori ancora maggiore – è questo il caso di tutte le rappresentazioni fotografiche o realistiche – le tavolozze non vengono usate; il colore è allora codificato direttamente nelle componenti cromatiche secondo il modello di colore adoperato.

Codifica Immagini (cenni)

MODELLO DI COLORE (Nota: modello di colore e spazio di colore sono concetti differenti. Tuttavia in questa sede non si mette in evidenza questa differenza)

- Un modello di colore è un modello matematico astratto che permette di rappresentare i colori in forma numerica, tipicamente utilizzando tre o quattro valori o componenti cromatiche (per esempio RGB e CMYK sono modelli di colore).
- Per esempio, quando i colori sono visualizzati sul display di un computer, sono definiti solitamente nello spazio di colore RGB (colori rosso, verde e blu). Posso definire l'intensità di ciascun 'canale di colore' associando per es un byte al rosso (R), un byte al verde (G), un byte al blu (B). Per es. 255,0,0 sarà il rosso 'puro'.
- CMYK (Cyan, Magenta, Yellow, BlacK) è un modello di colore detto anche di quadricromia. I colori ottenibili con la quadricromia (sintesi sottrattiva) sono un sottoinsieme della gamma visibile, quindi non tutti i colori che vediamo possono essere realizzati con la quadricromia, così come non tutti i colori realizzati con l'insieme RGB cioè quelli che vediamo sui nostri monitor (sintesi additiva) hanno un corrispondente nell'insieme CMYK.
- I colori dei quattro inchiostri usati in tipografia e nelle stampanti a colori sono proprio Cyan, Magenta, Yellow, Black. I primi tre colori (sovrapposti in diverse percentuali) possono dare origine quasi a qualunque altro colore. Ma non si riesce comunque ad ottenere il nero; sovrapponendoli si ottiene il bistro, colore simile a una tonalità di marrone molto scura. Perciò nei processi di stampa si è aggiunto l'inchiostro di un quarto colore per avere il nero pieno.

Compressione dell’informazione

- La compressione nel campo dell'informatica è una tecnica preposta **alla riduzione del numero di bit necessari** per immagazzinare un'informazione.

- Lo scopo è quello di **ridurre le dimensioni** occupate dai file sulle memorie di massa oppure **ridurre i tempi** di trasmissione dei dati (per esempio su Internet). Le varie tecniche di compressione cercano di organizzare in modo più efficiente le informazioni al fine di ottenere una memorizzazione che richieda minor uso di risorse.

Compressione dell'informazione

Le tecniche di compressione si dividono in due grandi categorie:

- lossy**: compressione** con perdita d'informazione
- lossless**: compressione** senza perdita d'informazione

In genere si può usare il termine 'messaggio' (messaggio da comprimere, messaggio compresso etc.) intendendo una qualsiasi sequenza di byte che veicola l'informazione di interesse. Le considerazioni che seguono si applicano indifferentemente a file o a sequenze di byte fisicamente memorizzate nella memoria principale, o trasmessi attraverso un canale di trasmissione. È possibile che i termini messaggio e file siano utilizzati come sinonimi, anche se chiaramente questo non è corretto in generale.

Compressione lossy

- Le tecniche di **compressione con perdita di informazione** ottengono delle compressioni molto elevate dei file **a scapito del contenuto informativo** del file stesso. Il file prima della compressione e il file dopo la decompressione sono simili ma non identici.
- Queste tecniche sono applicate ad immagini, file audio e filmati, e sfruttano il fatto che una perdita di qualità a volte non è apprezzabile o comunque può rappresentare un valido compromesso rispetto alla riduzione delle dimensioni.

Compressione lossy

Esempi di casi in cui una perdita di informazione (cioè di qualità) è accettabile

- File audio che sarà ascoltato su un impianto stereo in automobile
- Film o foto visualizzati su un monitor a 15 pollici
- Trasmissione della voce (servizi fonia). Comprimere il messaggio, e quindi dover inviare meno byte, significa poter inviare il messaggio senza attese. L'interlocutore non percepisce pause o 'scatti' nella conversazione. La perdita di informazione si traduce in una perdita di 'sfumature' della voce, che è comunque molto più accettabile di scatti o pause artificiali nella conversazione.

Compressione lossy

JPEG

- JPEG è l'acronimo di Joint Photographic Experts Group, un comitato ISO/CCITT che ha definito il primo standard internazionale di compressione per immagini a tono continuo, sia a livelli di grigio che a colori. Attualmente JPEG è lo standard di compressione delle immagini fotografiche più utilizzato.
- Le estensioni più comuni per questo formato sono: .jpg, .jpeg, .jfif, .JPG, .JPE.
- **Idea base**: pixel vicini fra loro, se di colori molto simili, possono essere rappresentati dallo stesso colore 'medio'. Perdo i particolari dell'immagine ma riduco anche notevolmente la quantità di byte necessari.

Compressione lossy

- Il formato jpeg è il formato più utilizzato per la memorizzazione di fotografie fruibili su WEB. Da notare che in caso di utilizzo professionale delle foto, necessità di ingrandimenti etc. anche la piccola perdita di informazione potrebbe non essere accettabile. Per la memorizzazione di immagini per uso professionale si preferisce quindi in formato TIFF

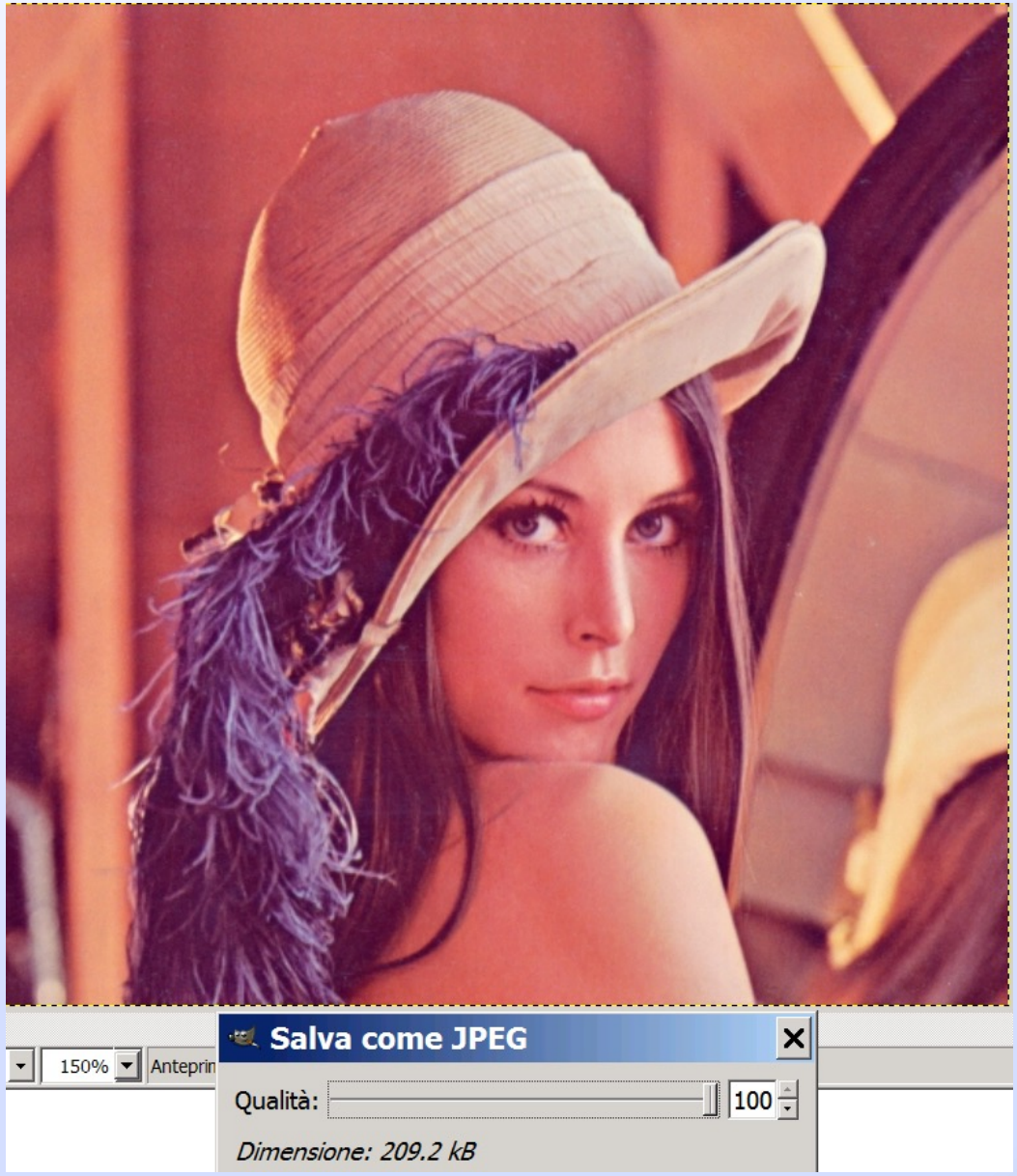
- Nota: in realtà il formato jpeg prevede la possibilità di utilizzare due tipi di compressione, uno con perdita di informazione, l'altro senza perdita di informazione ma studiato specificatamente per le immagini.

Compressione lossy

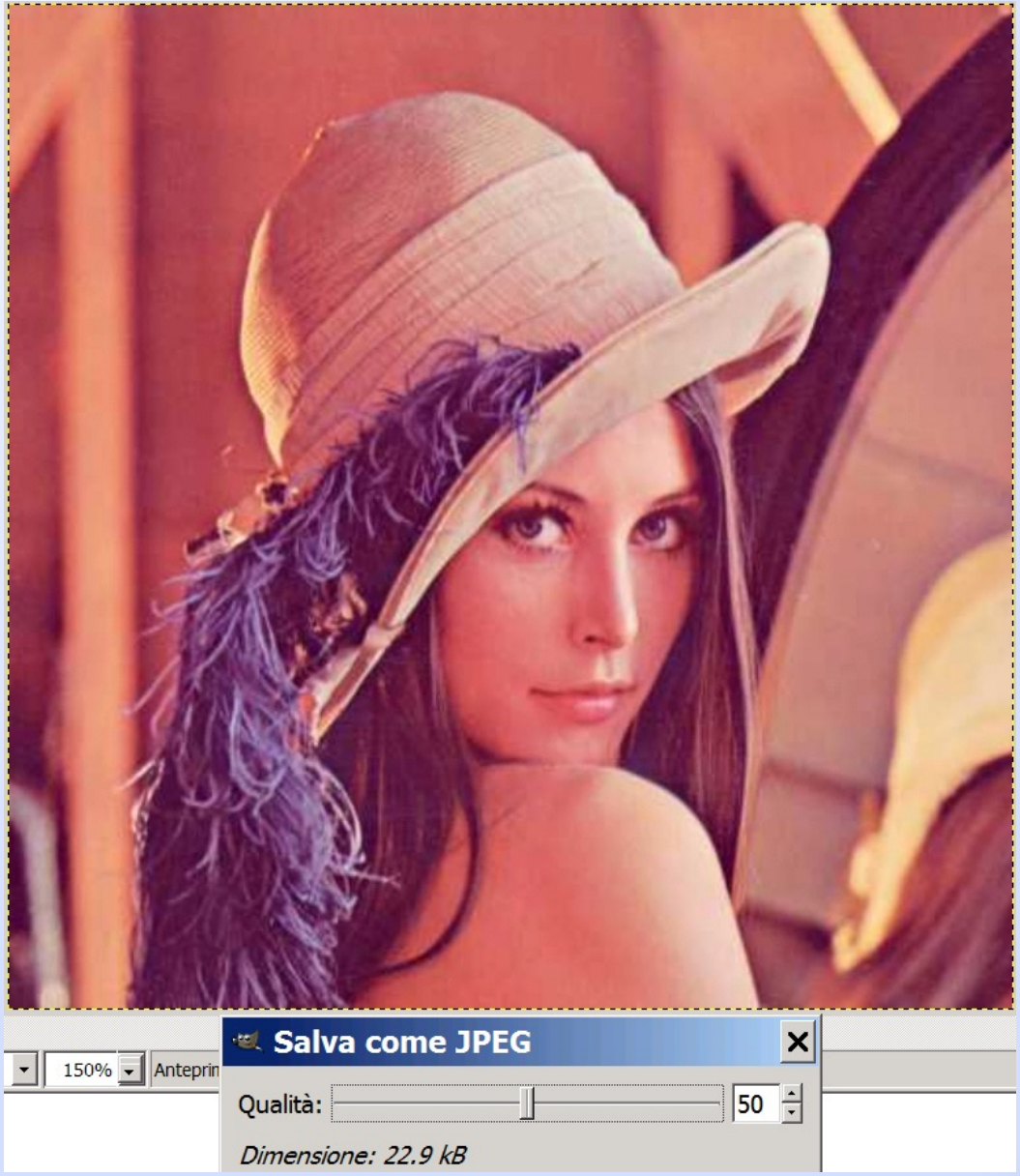
Lena Sjööblom (31 marzo 1951) ex modella svedese, playmate del mese di novembre 1972 della rivista Playboy.

Lena 512x512, 262144 pixel, RGB, 8 bit per colore
Dimensioni: 512x512*3=768 KByte

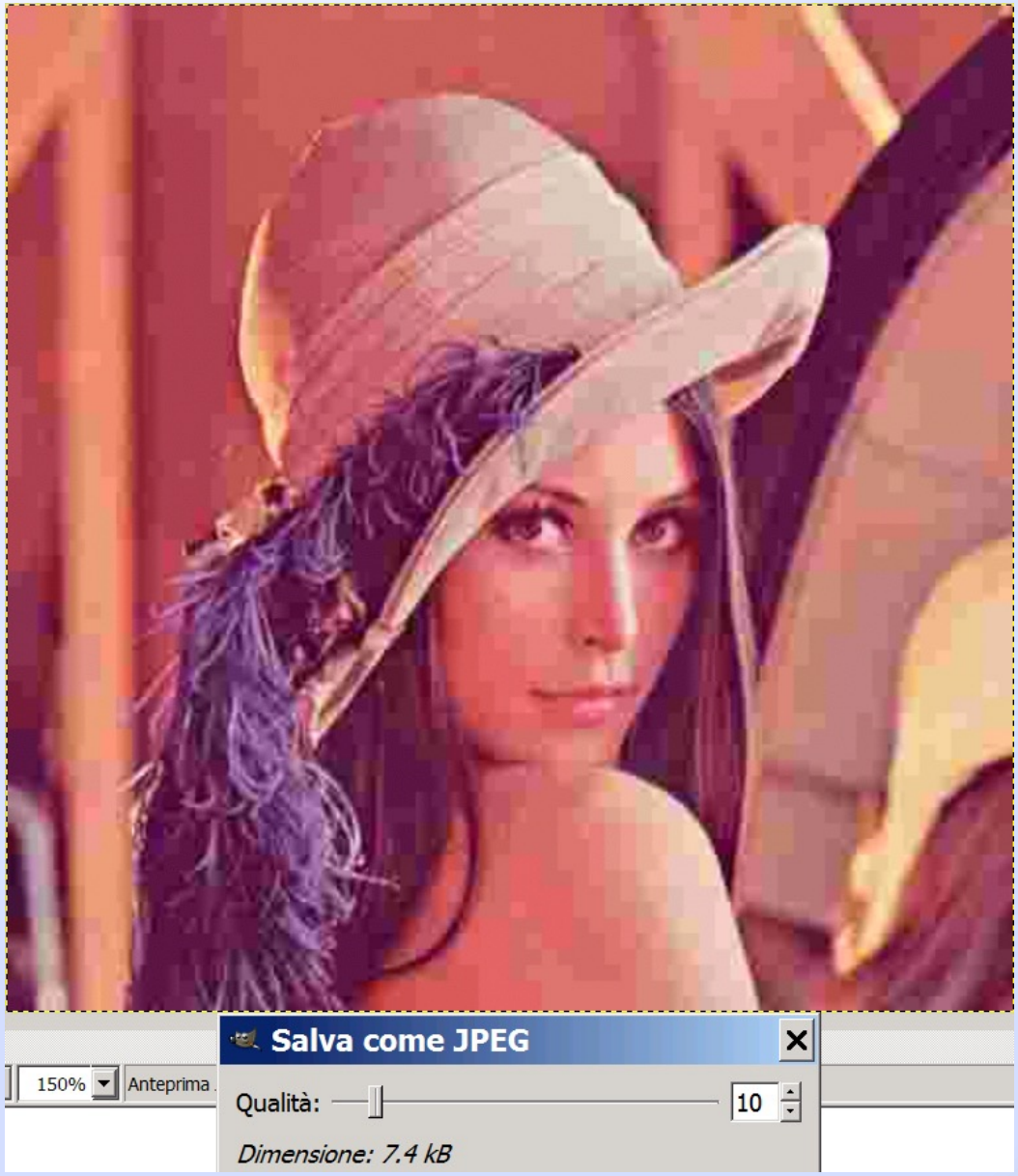
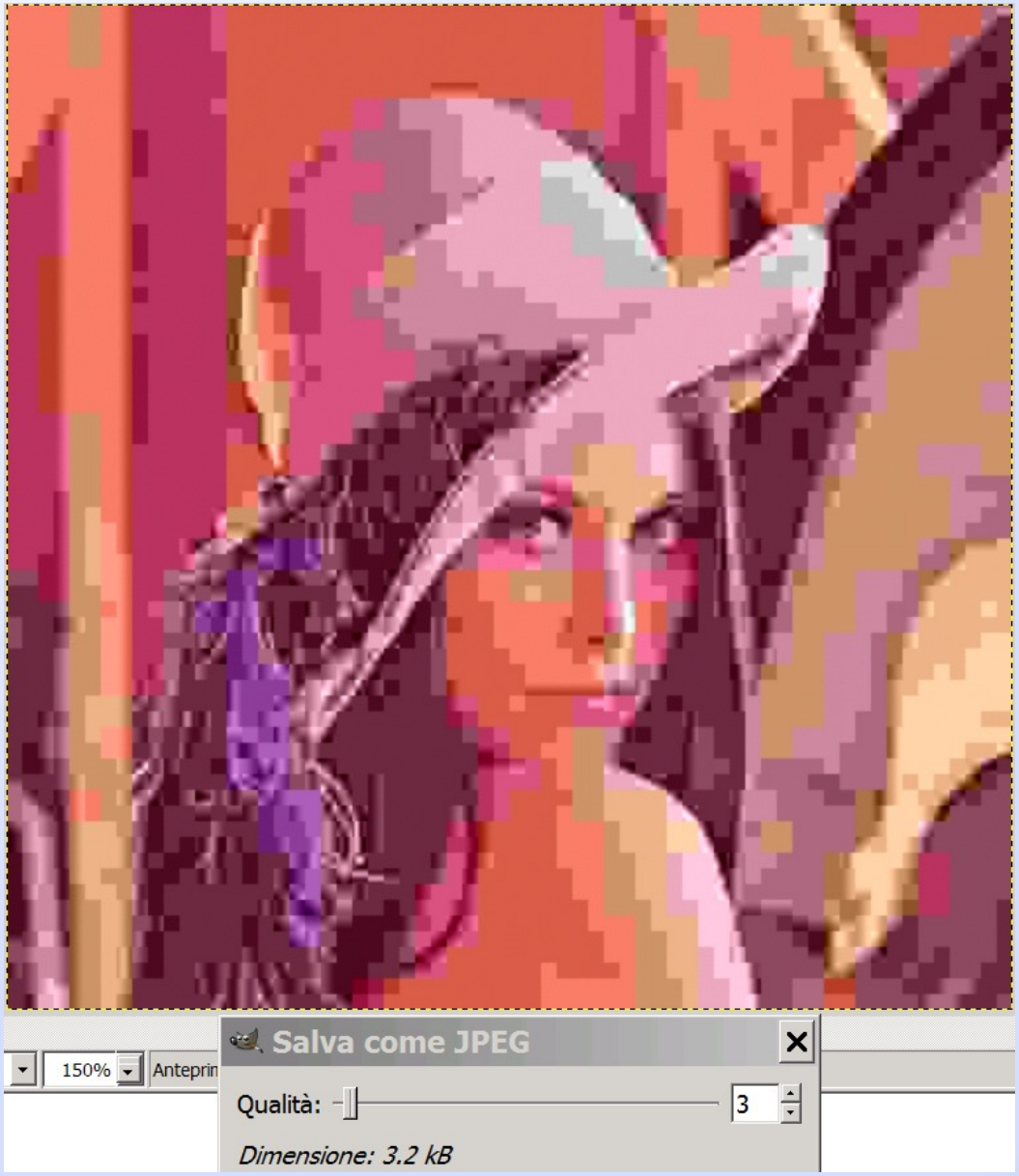
Compressione lossy



Compressione lossy



Compressione lossy



Compressione lossy